

3DGS²-TR: Scalable Second-Order Trust-Region Method for 3D Gaussian Splatting

Roger Hsiao¹ Yuchen Fang¹ Xiangru Huang¹ Ruilong Li² Hesam Rabeti² Zan Gojcic² Javad Lavaei¹
James Demmel¹ Sophia Shao¹

Abstract

We propose 3DGS²-TR, a second-order optimizer for accelerating the scene training problem in 3D Gaussian Splatting (3DGS). Unlike existing second-order approaches that rely on explicit or dense curvature representations, such as 3DGS-LM (Höllein et al., 2025) or 3DGS² (Lan et al., 2025), our method approximates curvature using only the diagonal of the Hessian matrix, estimated efficiently via Hutchinson’s method. Our approach is fully matrix-free and has the same complexity as ADAM (Kingma, 2014), $O(n)$ in both computation and memory costs. To ensure stable optimization in the presence of strong non-linearity in the 3DGS rasterization process, we introduce a parameter-wise trust-region technique based on the squared Hellinger distance, regularizing updates to Gaussian parameters. Under identical parameter initialization and without densification, 3DGS²-TR is able to achieve better reconstruction quality on standard datasets, using 50% fewer training iterations compared to ADAM, while incurring less than 1GB of peak GPU memory overhead (17% more than ADAM and 85% less than 3DGS-LM), enabling scalability to very large scenes and potentially to distributed training settings.

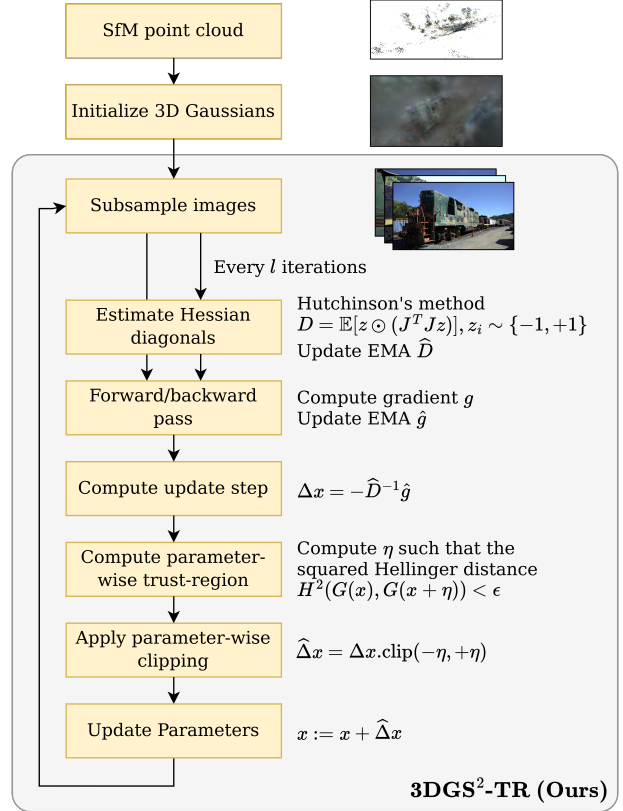


Figure 1. Overview of our proposed method.

1. Introduction

Recent advances in radiance field representations have revolutionized 3D content creation, enabling high-fidelity, photorealistic scene reconstruction from sparse input views. Neural Radiance Fields (NeRF) (Mildenhall et al., 2021) pioneered coordinate-based neural scene representations, achieving remarkable rendering quality but suffering from

¹University of California, Berkeley ²NVIDIA. Correspondence to: Roger Hsiao <roger_hsiao@berkeley.edu>.

slow training and inference times. To address these limitations, 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) proposed an explicit and efficient scene representation that models geometry and appearance using anisotropic 3D Gaussians optimized through differentiable rasterization. This formulation preserves view-dependent effects while enabling real-time rendering, rapidly establishing 3DGS as the new standard for real-time novel view synthesis—the task of rendering photorealistic images from previously unseen camera viewpoints given a limited set of input images. This capability is central to applications in virtual reality, augmented reality, robotics, and 3D content creation.

Building on the original 3DGS framework’s demonstra-

tion that anisotropic Gaussians coupled with GPU-friendly rasterization can achieve both high-quality synthesis and real-time performance (Kerbl et al., 2023), a number of subsequent works have proposed strategies to further improve rasterization efficiency. Recent efforts focus on accelerating forward rasterization and inference through optimized GPU kernels, memory access patterns, and tile-based rendering strategies, enabling efficient large-scale deployment and real-time visualization (Durvasula et al., 2023; Feng et al., 2025; Mallick et al., 2024; Ye et al., 2025; Papantonakis et al., 2024; Zhao et al., 2024).

Despite these advances in rasterization and scene representation, scene training in 3DGS remains a bottleneck in the pipeline. Although significantly faster than NeRF, 3DGS still requires training from scratch for each new scene, taking 20-40 minutes on commercial GPUs for commonly used datasets (Kerbl et al., 2023). This limits the applicability of 3DGS in large-scale or real-time exploration scenarios.

Several lines of research address this training bottleneck through orthogonal approaches. One direction targets the backward pass, which accumulates gradients into Gaussian splats with poor GPU utilization due to resource contention when many pixels share the same Gaussian primitives. Recent work has identified this as a dominant bottleneck and proposed alternative accumulation schemes or kernel redesigns to mitigate atomic contention (Durvasula et al., 2023; Ye et al., 2025). Another approach reduces training time through more compact scene representations. Related efforts improve the densification process or introduce pruning strategies to reduce the total number of Gaussian primitives while preserving reconstruction quality, thereby indirectly accelerating training speed (Kheradmand et al., 2024; Fang & Wang, 2024; Rota Bulò et al., 2024; Ali et al., 2024; Zhang et al., 2024b; Hanson et al., 2025b).

Our work aligns most closely with a third direction: improving the optimizer itself. The standard optimizer for 3DGS is ADAM (Kingma, 2014), a first-order gradient-descent method widely used in deep learning. While first-order optimizers are easy to implement and scale well, they suffer from slow convergence in highly non-convex and ill-conditioned parameter spaces (Sutton, 1986; Dauphin et al., 2014; Bottou et al., 2018). The optimization landscape of 3DGS is particularly challenging due to the strong coupling between geometry parameters (position, rotation, scale) and appearance parameters (opacity, color), leading to inefficient convergence and excessive training iterations. Additionally, first-order methods require meticulous tuning of learning rates for each parameter (Sutton, 1986; Schaul et al., 2013; Dauphin et al., 2014; Bottou et al., 2018).

3DGS occupies a unique position in machine learning: the model quality scales with the number of parameters, yet each parameter remains highly interpretable as a compo-

nent of an unnormalized 3D Gaussian distribution. This has motivated prior work to explore second-order optimization algorithms—such as Newton’s method, Gauss-Newton, or Levenberg-Marquardt—to achieve superlinear convergence (Höllein et al., 2025; Lan et al., 2025; Pehlivan et al., 2025). However, the 3DGS rasterization function poses fundamental challenges for second-order methods. First, it is highly nonlinear due to the sequential rendering of Gaussian splats at each pixel, where the transmittance seen by each splat depends on the opacities of all preceding splats in depth order. Second, the loss function is only piecewise continuous since depth-based sorting of Gaussian splats causes render order to change discontinuously with position parameters. These discontinuities become more frequent in regions with dense clusters of splats, which commonly occur towards the end of training. Moreover, existing implementations of second-order methods incur high memory overhead from storing curvature information and suffer from expensive per-iteration costs due to matrix operations, offsetting any convergence speedup and making them unsuitable for production use with large scenes.

In practice, second-order optimizers for 3DGS have yet to outperform first-order methods in either reconstruction quality or convergence speed, leading to limited adoption. However, recent work in deep learning, such as Sophia (Liu et al., 2023), introduces a lightweight, second-order approach that utilizes a diagonal Hessian estimate to account for loss surface curvature. By adapting step sizes to sharp or flat regions with minimal computational overhead, Sophia achieves faster convergence and superior stability in large-scale non-convex tasks such as large language model training. Nevertheless, the performance of Sophia and its variants remains largely unexplored in computer vision applications.

Based on these observations, we propose the following three principles to accelerate scene training in 3DGS.

1. *Cheap per-iteration computation.* Since 3DGS rasterization is highly nonlinear with frequent discontinuities, we favor many small, inexpensive steps over a few large, expensive ones that would likely violate local linear or quadratic approximations. Given that the backward pass is the primary training bottleneck, our strategy introduces modest computational overhead in pre- or post-processing the update step to accelerate convergence and reduce the total number of backward passes required.
2. *Parameter-linear memory scaling.* Persistent storage across iterations should scale with the number of Gaussian parameters, not with the number of pixels, which often exceed the parameter count by orders of magnitude for large scenes with high-resolution training images.

3. *Trust region constraints.* Update step sizes should be bounded by a trust region that limits the impact of nonlinear interactions between Gaussian splats. Specifically, we bound the change in transmittance experienced by each splat before and after the update, ensuring steps remain within the region of validity for local approximation.

Guided by these principles, we present 3DGS²-TR, a second-order optimizer for accelerated 3DGS scene training. We summarize our main contributions as follows:

1. We apply a second-order update rule that uses Hessian diagonals to 3DGS training, which has the same $O(n)$ complexity as ADAM in both computation and memory.
2. We propose an effective trust region for Gaussian parameter updates that bounds the squared Hellinger distance of each Gaussian splat before and after optimization steps, providing a principled constraint on geometric changes and requiring only a single tunable hyperparameter.

2. Related Work

2.1. Novel View Synthesis

Novel view synthesis aims to generate photorealistic images from arbitrary viewpoints given a set of input images. Neural Radiance Fields (NeRF) (Mildenhall et al., 2021) pioneered implicit scene representations using multilayer perceptrons to encode volumetric density and view-dependent appearance, achieving photorealistic results through volumetric ray marching but suffering from slow training and inference. In contrast, 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) models scenes explicitly as collections of anisotropic 3D Gaussians, enabling real-time rendering through efficient differentiable rasterization while maintaining high visual fidelity. This has established 3DGS as the state-of-the-art for real-time novel view synthesis, inspiring extensive research on improving rendering quality (Lu et al., 2024; Yu et al., 2024), scaling to larger environments (Kerbl et al., 2024; Song et al., 2024), and enhancing geometric accuracy.

2.2. Accelerating 3DGS Training and Rendering

While 3DGS achieves real-time rendering, training remains a bottleneck for large-scale scenes. Recent work addresses this through complementary strategies.

Compact representations. Several methods aim to reduce the number of Gaussian primitives while preserving quality. EAGLES (Girish et al., 2024) uses quantized embeddings and coarse-to-fine training. LightGaussian (Fan et al., 2024)

prunes low-contribution Gaussians and compresses spherical harmonics. C3DGS (Lee et al., 2024) learns binary masks to remove redundant primitives, while Speedy-Splat (Hanson et al., 2025a) introduces tight bounding boxes and dual pruning strategies.

Rasterization optimizations. AdR-Gaussian (Wang et al., 2024) culls low-opacity Gaussian-tile pairs and balances workloads. FlashGS (Feng et al., 2025) and gsplat (Ye et al., 2025) optimize CUDA kernels and memory access patterns. Additional improvements include modified densification heuristics (Fang & Wang, 2024; Kheradmand et al., 2024) that reduce Gaussian proliferation.

2.3. Second-Order Optimizers

Recent research in large-scale machine learning explores replacing ADAM (Kingma, 2014) with second-order methods to accelerate convergence. Classical second-order methods—Newton’s method, Gauss-Newton, and Levenberg-Marquardt—achieve superlinear convergence via Hessian curvature information but face $O(n^2)$ memory and $O(n^3)$ computational costs. Lightweight alternatives such as Ada-Hessian (Yao et al., 2021) and Sophia (Liu et al., 2023), instead use diagonal Hessian approximations for efficient large-scale optimization, avoiding the storage and computation overhead while gaining improvement in performance.

Applications of second-order optimizers have been explored in 3DGS as well. For example, 3DGS-LM (Höllein et al., 2025) integrates the Levenberg-Marquardt method, while 3DGS2 (Lan et al., 2025) partitions parameters and solves small Newton systems in sequence. Recently, (Pehlivan et al., 2025) adopted a matrix-free design, applying the preconditioned conjugate gradient method to solve the system, and introduced a pixel sampling strategy based on residuals.

However, some fundamental challenges have not been adequately addressed: 3DGS rasterization is highly nonlinear and only piecewise continuous due to depth-based sorting, violating smoothness assumptions of classical second-order methods. Moreover, the high parameter and pixel counts make it intractable to materialize full matrices, necessitating careful camera perspective grouping and image subsampling (Höllein et al., 2025; Lan et al., 2025). Consequently, existing approaches only work well on small scenes or when initialized near a sufficiently good local minimum. In contrast, our work adapts Sophia’s lightweight framework to 3DGS with modifications tailored to Gaussian-based representations while maintaining efficiency and broad applicability.

3. Background and Notations

3.1. Review of Gaussian Splatting

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) represents a scene as a collection of 3D Gaussians splats, each parametrized by its position μ , scale S , rotation R , opacity α , and color $\mathcal{C}$. We can write each Gaussian as an unnormalized Gaussian distribution

$$G(z) = \alpha \mathcal{C} \exp\left(-\frac{1}{2}(z - \mu)^T \Sigma^{-1}(z - \mu)\right) \quad (1)$$

$$= \alpha \mathcal{C} (2\pi)^{\frac{3}{2}} \det(\Sigma)^{\frac{1}{2}} \mathcal{N}(z; \mu, \Sigma) \quad (2)$$

The covariance is split into separate rotation and scaling parameters as $\Sigma = R^T S^T S R$. $\mathcal{C}$ is the view-dependent color modeled using spherical harmonic coefficients of order 3. To render an image of size $H \times W$ from a given viewpoint, all Gaussians are first projected into 2D Gaussian splats via a tile-based differentiable rasterizer. The projected splats are then α -blended along each camera ray to obtain the color C_ω at pixel ω :

$$C_\omega = \sum_{i \in \mathcal{K}} C_i \bar{\alpha}_i T_i, \text{ with } T_i = \prod_{j=1}^{i-1} (1 - \bar{\alpha}_j),$$

where $\mathcal{K}$ denotes the set of Gaussian kernels of size $|\mathcal{K}|$, C_i is the color of the i -th splat along the ray, $\bar{\alpha}_i$ is the 2D Gaussian’s evaluated opacity, and T_i represents transmittance. The complete Gaussian parameter vector x concatenates the parameters of all $|\mathcal{K}|$ kernels by groups, (e.g. $x = (x_{\text{position}}, x_{\text{scaling}}, x_{\text{rotation}}, x_{\text{opacity}}, x_{\text{color}})$). To fit the Gaussian parameter x , 3DGS minimizes the discrepancy between the rendered and ground-truth images. At each pixel ω , the loss function is written as

$$\mathcal{L}_\omega = (1 - \lambda) \mathcal{L}_{\omega, L_1} + \lambda \mathcal{L}_{\omega, D-SSIM}. \quad (3)$$

The L1 and SSIM losses can each be further split into three components, one for each RGB channel. The total loss is the mean of the loss over all pixels and all channels.

3.2. Optimization Problem Formulation

Let $x \in \mathbb{R}^n$ be the parameter vector by concatenating the parameters of all Gaussian kernels. Let M be the number of training images. Each training image emits $6 \times H \times W$ loss components with a total of m loss components over all images. We use $f_i(x)$ to denote the vector loss function for image i , where each entry is the square root of a loss component, so that the 3DGS scene training problem can be formulated as a nonlinear least-squares problem

$$\min_x f(x) := \frac{1}{2m} \left\| \begin{bmatrix} f_1(x)^T & \cdots & f_M(x)^T \end{bmatrix}^T \right\|_2^2. \quad (4)$$

The full Jacobian matrix is denoted as $J(x) = \begin{bmatrix} J_1(x)^T & \cdots & J_M(x)^T \end{bmatrix}^T$ and the pseudo-Hessian matrix is $H(x) = J(x)^T J(x) = \sum_{i=1}^M J_i(x)^T J_i(x)$.

Algorithm 1 3DGS²-TR

Require: Initial parameters $x_1 \in \mathbb{R}^n$, Hessian diagonals update interval $l \in \mathbb{N}$, EMA decay rates $\theta_1, \theta_2 \in (0, 1)$, trust-region parameter $\epsilon > 0$, Hutchinson sample size $\nu \in \mathbb{N}$.

- 1: Set $\hat{g}_0 = 0, \hat{D}_{1-l} = 0$.
- 2: **for** $t = 1$ to T **do**
- 3: Sample a mini-batch $\mathcal{S}_1$ of images and compute the stochastic gradient g_t .
- 4: Update EMA $\hat{g}_k = \theta_1 \hat{g}_{k-1} + (1 - \theta_1) g_k$.
- 5: **if** $t \bmod l = 1$ **then**
- 6: Sample a mini-batch $\mathcal{S}_2$ of images and compute $D_t = \text{Hutch}(x_t, \mathcal{S}_2, \nu)$.
- 7: Update EMA $\hat{D}_t = \theta_2 \hat{D}_{t-l} + (1 - \theta_2) D_t$.
- 8: **else**
- 9: $\hat{D}_t = \hat{D}_{t-1}$.
- 10: **end if**
- 11: Compute the update step $\Delta x_t = -\hat{D}_t^{-1} \hat{g}_t$.
- 12: Compute the parameter-wise trust-region radius

$$\eta = \text{SHD}(x_t, \epsilon).$$

- 13: Apply parameter-wise clipping

$$\hat{\Delta} x_t = \Delta x_t \cdot \text{clip}(-\eta, +\eta).$$

- 14: Update the parameters $x_{t+1} = x_t + \hat{\Delta} x_t$.
- 15: **end for**

4. Methodology

3DGS²-TR is a scalable second-order optimization framework for solving the 3DGS training objective (4). Our method adopts a stochastic Gauss–Newton formulation and incorporates curvature information while remaining memory- and compute-efficient. An overview of the proposed pipeline is illustrated in Figure 1, whereas the full algorithm is summarized in Algorithm 1.

4.1. Algorithm Design

We initialize the 3D Gaussian primitives using a sparse point cloud obtained from structure-from-motion (SfM). Given the parameters x_t at iteration t , we first subsample a mini-batch of images $\mathcal{S}_1$ to estimate the stochastic gradient

$$g_t = \frac{1}{m} \frac{M}{|\mathcal{S}_1|} \sum_{i \in \mathcal{S}_1} J_i(x_t)^T f_i(x_t),$$

where $f_i(x_t)$ denotes the vector loss and $J_i(x_t)$ is its Jacobian of the i -th image with respect to the Gaussian parameters x_t . To reduce gradient noise and improve stability, we maintain an exponential moving average (EMA) of the

gradients with decay rate $\theta_1 \in (0, 1)$,

$$\hat{g}_t = \theta_1 \hat{g}_{t-1} + (1 - \theta_1) g_t.$$

To incorporate second-order information while maintaining $O(n)$ memory and computation costs, we follow the spirit of Sophia (Liu et al., 2023) and estimate the diagonal of the Gauss-Newton matrix $J^T J$ using Hutchinson’s method. Specifically, every l iterations, we subsample another mini-batch of images $\mathcal{S}_2$ and compute

$$\begin{aligned} D_t &= \text{Hutch}(x_t, \mathcal{S}_2, \nu) \\ &= \frac{1}{m} \frac{M}{|\mathcal{S}_2|} \frac{1}{\nu} \sum_{i=1}^{\nu} z^{(i)} \odot \left(\sum_{j \in \mathcal{S}_2} J_j(x_t)^T J_j(x_t) z^{(i)} \right), \end{aligned}$$

where $\{z^{(i)}\}_{i=1}^m$ are independent Rademacher random vectors and $\odot$ denotes element-wise multiplication. The scaling factors $M|\mathcal{S}_1|^{-1}$, $M|\mathcal{S}_2|^{-1}$ are required to maintain an unbiased estimate of the gradient and the Hessian diagonal after sampling. Such image subsampling and diagonal approximation significantly reduce computational and memory overhead, since the Hessian-vector product $J_j(x_t)^T J_j(x_t) z^{(i)}$ corresponds to exactly one forward and backward pass on a training image. To further stabilize curvature estimates, we maintain an EMA of D_t with decay rate θ_2 and reuse the previous estimate in intermediate iterations, i.e.,

$$\hat{D}_t = \begin{cases} \theta_2 \hat{D}_{t-1} + (1 - \theta_2) D_t & \text{if } t \bmod l = 1, \\ \hat{D}_{t-1} & \text{otherwise.} \end{cases}$$

Given $\hat{D}_t$ and $\hat{g}_t$, we compute the update step

$$\Delta x_t = -\hat{D}_t^{-1} \hat{g}_t.$$

To improve robustness under strong nonlinearity and discontinuities in the 3DGS rasterization process, we apply a parameter-wise trust-region constraint. Specifically, we compute a trust-region radius for each parameter

$$\eta = \text{SHD}(x_t, \epsilon),$$

based on the squared Hellinger distance, which bounds the change of each Gaussian primitive induced by the update. Details of this construction are provided in Section 4.2.

The final update is obtained by parameter-wise clipping,

$$\hat{\Delta} x_t = \Delta x_t \cdot \text{clip}(-\eta, +\eta),$$

followed by the parameter update

$$x_{t+1} = x_t + \hat{\Delta} x_t. \quad (5)$$

The minibatch sizes are selected based on the tradeoff between algorithmic performance and computational cost. In our experiments, we set $|\mathcal{S}_1| = |\mathcal{S}_2| = \nu = 1$ and $l = 10$, which corresponds to an approximately 10% overhead compared to ADAM.

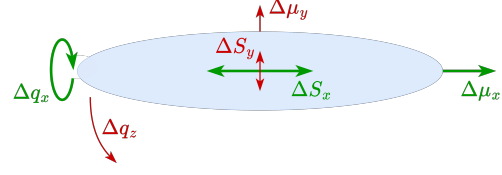


Figure 2. Example scene with a single elongated Gaussian splat. The x -axis and y -axis lie on the page; the z -axis comes out of the page. The green arrows mark the directions in which the Gaussian has more freedom; while the red arrows denote otherwise.

4.2. Trust-Region Based on Squared Hellinger Distance

As in Sophia (Liu et al., 2023), individual update steps can occasionally become excessively large due to the high variance of Hutchinson’s estimator. This issue is further exacerbated in 3DGS by the highly nonlinear and piecewise-continuous nature of the rasterization process, particularly in later optimization stages where dense clusters of Gaussian primitives frequently emerge.

In ADAM (Kingma, 2014), this instability is typically addressed through careful learning-rate tuning and scheduled decay of the position updates. On the other hand, 3DGS-LM (Höllein et al., 2025) mitigates nonlinearity by introducing an adaptive regularization term that effectively interpolates between Gauss-Newton and gradient descent updates. However, both approaches operate at a global or group level and do not exploit the strong interpretability of individual Gaussian parameters.

We instead propose a *parameter-wise trust-region* strategy that leverages the explicit geometric and photometric meaning of each parameter in 3DGS. Since interactions between Gaussian splats primarily occur through their projected opacity $\bar{\alpha}$ on the image plane, a natural trust-region design should directly limit changes to this quantity across viewpoints.

To build intuition, consider a scene containing a single anisotropic Gaussian (see Figure 2). Intuitively, the Gaussian should translate or expand more conservatively along its short axis to avoid abrupt changes in rendered pixels. Additionally, rotations about the long axis should be less restrictive than those about the short axis. Finally, more transparent Gaussians should be allowed to evolve more rapidly than highly opaque ones. All of these constraints can be formalized by constructing parameter-wise trust regions that bound the discrepancy between the Gaussian splat before and after an update.

Squared Hellinger distance. We quantify the magnitude of an update to a Gaussian G using the squared Hellinger distance

$$H^2(G, G') = \frac{1}{2} \int \left(\sqrt{G(z)} - \sqrt{G'(z)} \right)^2 dz. \quad (6)$$

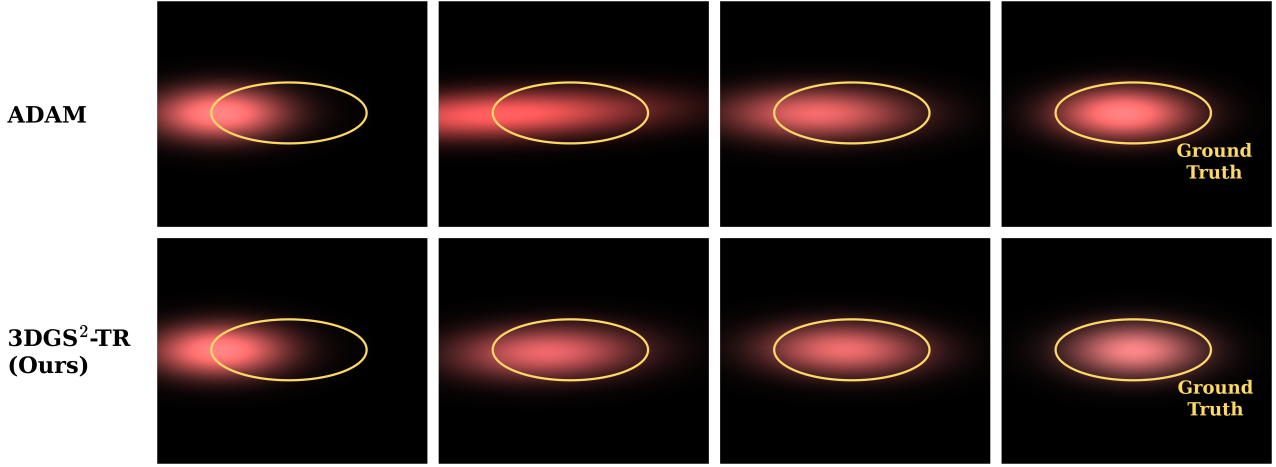


Figure 3. A single Gaussian fitting example where the Gaussian splat is initialized with a small perturbation. Images from left to right show the progression of optimization to fit the perturbed splat to the ground truth splat (denoted with the orange circle). Both ADAM and 3DGS²-TR eventually recover the Gaussian parameters close to the ground truth, but 3DGS²-TR deforms the Gaussian much less due to the trust-region bound, which increases the stability of training.

For unnormalized Gaussian primitives $G = Z \cdot \mathcal{N}(z; \mu, \Sigma)$ and $G' = Z' \cdot \mathcal{N}(z; \mu', \Sigma')$ with probability mass Z, Z' , $H^2(G, G')$ emits a closed-form solution

$$H^2(G, G') = \frac{1}{2}(Z + Z') - (Z \cdot Z')^{\frac{1}{2}} \cdot \Sigma_1 \cdot \exp\left(-\frac{\Delta\mu^T \Sigma_2^{-1} \Delta\mu}{8}\right),$$

where $\Delta\mu = \mu - \mu'$, $\Sigma_1 = \frac{\det(\Sigma)^{\frac{1}{4}} \det(\Sigma')^{\frac{1}{4}}}{\det(\frac{\Sigma + \Sigma'}{2})^{\frac{1}{2}}}$, and $\Sigma_2 = \frac{\Sigma + \Sigma'}{2}$.

In the 3DGS setting, the probability mass satisfies $Z = \alpha \mathcal{C} \det(\Sigma)^{1/2}$ and $Z' = \alpha' \mathcal{C}' \det(\Sigma')^{1/2}$. While alternative divergence measures (e.g., KL divergence) could also be employed, we find that the squared Hellinger distance yields trust-region bounds that are both simple to compute and intuitively interpretable, being the integral of the coordinate-wise difference between two Gaussian splats.

Scale normalization. Because the apparent mass of a rendered Gaussian can be arbitrarily scaled by moving along the viewing direction, we normalize the squared Hellinger distance by the determinant of the scale matrix. Specifically, we rescale $H^2(G, G')$ by $\det(\Sigma)^{-1/2} = \det(S)^{-1}$, recalling that $\Sigma = R^T S^T S R$ with orthogonal rotation matrix R . This normalization can be interpreted as comparing Gaussians at a fixed effective distance to the camera. For all parameters except color, we treat the opacity α as the total mass of the Gaussian, as only the opacity component affects future Gaussians to be rendered.

Parameter-wise trust-region radius. We now derive a parameter-wise trust-region radius $\eta = \text{SHD}(x_t, \epsilon)$ such that, when a single parameter is perturbed while all others remain fixed, the normalized squared Hellinger distance between the Gaussian before the update $G(x_t)$ and after the update $G(x_t + \eta)$ is bounded by ϵ :

$$\eta = \text{SHD}(x_t, \epsilon) := \arg \max_{|\eta|} \left\{ |\eta| : \frac{H^2(G, G')}{\det(S)} < \epsilon \right\}.$$

Below, we summarize the resulting trust-region bounds for each parameter type; detailed derivations are deferred to [Appendix B](#).

Bound on the mean $\mu = (\mu_x, \mu_y, \mu_z)$. For perturbations $\Delta\mu = (\Delta\mu_x, \Delta\mu_y, \Delta\mu_z)$, with $c \in \{x, y, z\}$, we require

$$|\Delta\mu_c| < \sqrt{-8 \Sigma_{cc} \ln\left(1 - \frac{\epsilon}{\alpha}\right)}.$$

Bound on the scale matrix S . Let

$$S = \begin{bmatrix} S_x & & \\ & S_y & \\ & & S_z \end{bmatrix}, \quad S' = \begin{bmatrix} S'_x & & \\ & S'_y & \\ & & S'_z \end{bmatrix},$$

and define $\Delta S = S' - S$ with blockwise differences $\Delta S_x, \Delta S_y, \Delta S_z$, then with $c \in \{x, y, z\}$, we require

$$|\Delta S_c| < \sqrt{\frac{4 S_c^2 \epsilon}{\alpha}}.$$

Bound on the opacity α . Let $\alpha' = \alpha + \Delta\alpha$. To control the mass discrepancy, we require

$$|\Delta\alpha| < \sqrt{8\alpha\epsilon}.$$

Bound on the color $\mathcal{C}$. Let $\mathcal{C} = \mathcal{C}_r + \mathcal{C}_g + \mathcal{C}_b$, $\mathcal{C}' = \mathcal{C}'_r + \mathcal{C}'_g + \mathcal{C}'_b$ and $\Delta\mathcal{C}_c = \mathcal{C}'_c - \mathcal{C}_c$ for $c \in \{r, g, b\}$. We impose

$$|\Delta\mathcal{C}_c| < \sqrt{\frac{8\mathcal{C}_c \epsilon}{\alpha}}.$$

Bound on the rotation R . The rotation matrix R is parameterized by a quaternion (q_x, q_y, q_z, q_w) with squared norm $q^2 = q_x^2 + q_y^2 + q_z^2 + q_w^2$. Let $\beta_c > 0$ denote a geometry-dependent constant that upper bounds the sensitivity of the induced Gaussian displacement with respect to perturbations in the c -th quaternion component (see [Appendix B](#)). To ensure the rotation update remains within the trust region, for $c \in \{x, y, z, w\}$ it suffices to require

$$|\Delta q_c| < \sqrt{-\frac{8}{\beta_c} \ln\left(1 - \frac{\epsilon}{\alpha}\right)}.$$

We illustrate the effects of the proposed Hellinger-distance-based trust region using a single Gaussian fitting example in [Figure 3](#).

5. Evaluations

We now present the evaluation results of our method. The forward-mode automatic differentiation of the 3DGS rasterization kernel uses an in-house CUDA implementation based on duo numbers, which can be found at <https://github.com/rogerhh/diff-gaussian-rasterization-jvp>. The parameter-wise trust-region radius is implemented as a custom CUDA kernel. The training pipeline and the Sophia optimizer is implemented in PyTorch following the original 3DGS implementation.

Our experiments are run on A100-SXM4 GPUs with 6921 CUDA cores and 80 GB VRAM. We evaluate our method on the same datasets as 3DGS, namely, all scenes from MiP-NeRF360, two scenes from Tanks & Temples, and two scenes from Deep Blending ([Barron et al., 2022](#); [Knapitsch et al., 2017](#); [Hedman et al., 2018](#)). For each dataset, we initialize the 3D Gaussian splats with the standard SfM point cloud. As our diagonal estimator currently does not handle inserting new diagonal entries, all of our experiments are performed *without densification*.

In all experiments, we set the Hessian diagonal update interval to $l = 10$ to balance computational cost and curvature accuracy, following the default configuration of Sophia ([Liu et al., 2023](#)), and choose the exponential moving average (EMA) parameters $\theta_1 = 0.9$ and $\theta_2 = 0.999$. The trust-region radius ϵ follows an exponential decay schedule from 10^{-6} to 10^{-8} over the course of training.

For our baselines, we compare our method against ADAM, the SOTA first-order method, and 3DGS-LM, the only

second-order optimizer for 3DGS with open-source implementation.

We also include an ablation study which applies our Hellinger-distance-based trust region to the ADAM update step (ADAM-TR). On the other hand, Sophia optimizer alone without trust region tends to produce NaN values due to instability of the stochastic diagonal estimator for some splats; therefore, our method is always reported with trust region enabled.

Since 3DGS-LM has a high computation overhead, we only run it for 150 iterations in total, and report the results at 35, 75, and 150 iterations. We run all other methods for 30k iterations and report results at 7k, 15k, and 30k iterations. We use the same train/test split and report the same metrics (SSIM, PSNR, LPIPS) on the test images as proposed by 3DGS ([Kerbl et al., 2023](#)).

The main quantitative results are presented in [Table 1](#). (Results for individual scenes can be found in [Appendix A](#).) Qualitative comparisons for some scenes are shown in [Figure 4](#). 3DGS²-TR significantly outperforms other methods, reaching comparable or better reconstruction quality using 50% fewer iterations than first-order methods (ADAM and ADAM-TR). In the case of Tanks & Temples, 3DGS²-TR is able to exceed the best ADAM PSNR by 0.56dB at 7k iterations and 1.19dB at 30k iterations. Moreover, 3DGS²-TR requires less than 1GB of additional GPU memory for training in all scenes, which is 17% more than ADAM and 85% less than 3DGS-LM on average.

In contrast, 3DGS-LM does not produce comparable results with the same initialization, given significantly more time and resources. We also note that by applying our trust-region clipping to the ADAM updates (ADAM-TR), we are already able to achieve better quality than the vanilla ADAM optimizer.

6. Discussions and Limitations

Our method achieves much faster convergence per step compared to ADAM. However, due to a naive implementation of the Sophia optimizer in PyTorch, our second-order method has a higher run time per iteration, which is not a fundamental limitation of the algorithm itself. We plan to optimize the PyTorch implementation to reach comparable performance to ADAM for tensor manipulation in future work.

Compared to other proposed second-order optimizers, our method is fairly non-intrusive to the vanilla training pipeline, which makes it amenable to orthogonal improvements to 3DGS training, such as Markov Chain Monte Carlo (MCMC) densification ([Kheradmand et al., 2024](#)), co-regularization ([Zhang et al., 2024a](#)), drop-out Gaussians ([Park et al., 2025](#)), etc. However, one current limitation with

Table 1. The main quantitative result. Best results are **boldfaced** and second best are underlined.

Method	SSIM ↓			PSNR ↑			LPIPS ↓			Time (s)	Peak GPU Mem (GB)
	@7k	@15k	@30k	@7k	@15k	@30k	@7k	@15k	@30k		
mipnerf											
ADAM	0.667	0.679	0.684	24.45	24.85	25.08	0.406	0.400	0.420	212	5.60
3DGS-LM ¹	0.493	0.520	0.549	20.02	21.02	21.88	0.598	0.577	0.548	2602	47.75
ADAM-TR (Ours)	<u>0.674</u>	<u>0.688</u>	<u>0.693</u>	<u>24.54</u>	<u>25.02</u>	<u>25.21</u>	<u>0.394</u>	<u>0.388</u>	<u>0.411</u>	<u>235</u>	<u>5.64</u>
3DGS ² -TR (Ours) ²	0.682	0.692	0.696	24.80	25.19	25.39	0.390	0.385	0.402	652	6.51
deepblending											
ADAM	0.845	0.854	0.857	<u>26.65</u>	27.18	27.36	0.372	0.365	0.387	180	5.42
3DGS-LM ¹	0.810	0.833	0.845	23.72	24.89	25.62	0.437	0.405	0.385	2235	33.33
ADAM-TR (Ours)	<u>0.855</u>	<u>0.866</u>	0.869	26.56	<u>27.34</u>	<u>27.58</u>	<u>0.357</u>	<u>0.351</u>	<u>0.375</u>	<u>195</u>	<u>5.51</u>
3DGS ² -TR (Ours) ²	0.859	0.867	<u>0.869</u>	26.91	27.54	27.74	0.354	0.349	0.367	600	6.26
tandt											
ADAM	0.723	0.755	0.766	20.69	21.41	21.66	0.309	0.298	0.341	201	3.17
3DGS-LM ¹	0.661	0.717	0.749	20.00	21.09	21.81	0.429	0.366	0.331	2105	25.53
ADAM-TR (Ours)	<u>0.774</u>	<u>0.791</u>	<u>0.798</u>	<u>21.62</u>	<u>22.16</u>	<u>22.42</u>	<u>0.273</u>	<u>0.264</u>	<u>0.295</u>	<u>227</u>	<u>3.27</u>
3DGS ² -TR (Ours) ²	0.787	0.800	0.805	22.12	22.63	22.85	0.262	0.255	0.279	484	4.12

¹ 3DGS-LM results are reported @35, @75, and @150 iterations.

² System implementation is not fully optimized. See Section 6 for details.

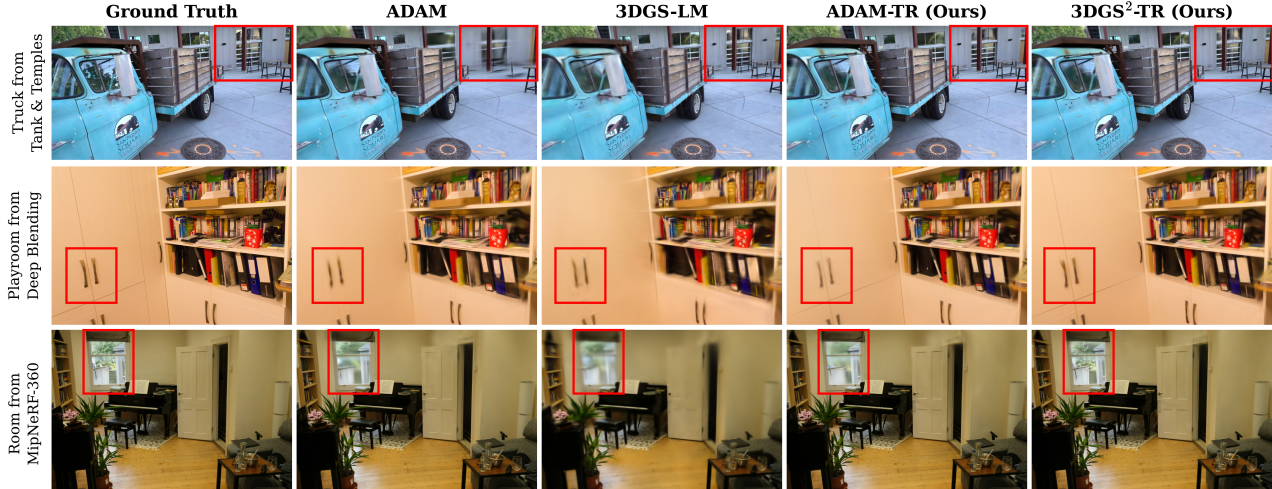


Figure 4. Qualitative comparison of different methods of the truck, playroom, and room scenes. The red boxes highlight regions where 3DGS²-TR significantly outperforms other methods.

our work is that we do not support adding or moving around Gaussian splats. Naively inserting rows and columns to the Hessian matrix results in a biased estimation of the diagonal entries, leading to degraded performance, which we aim to resolve in future work.

7. Conclusion

We present 3DGS²-TR, a second-order optimizer for training 3D Gaussian Splatting scenes. We approximate the

curvature information of the loss function using a stochastic diagonal estimator, which eliminates the storage and computation overhead of classical second-order methods, thus maintains the same $O(n)$ complexity as ADAM in both computational cost and memory. Furthermore, we introduce a parameter-wise trust-region technique based on the squared Hellinger distance to bound the update step size. Instead of tuning learning rates for each parameter group, our method requires only one hyperparameter. We show that 3DGS²-TR achieves better reconstruction quality after

50% of training iterations, while incurring only 10% of computation overhead and less than 1GB of memory overhead compared to ADAM. Our method is a drop-in replacement for the ADAM optimizer to accelerate 3DGS training in all settings.

Acknowledgements

We thank the reviewers for their valuable feedback and comments. This research was partially funded by SLICE Lab industrial sponsors.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ali, M. S., Qamar, M., Bae, S.-H., and Tartaglione, E. Trimming the fat: Efficient compression of 3d gaussian splats through pruning. *arXiv preprint arXiv:2406.18214*, 2024.
- Barron, J. T., Mildenhall, B., Verbin, D., Srinivasan, P. P., and Hedman, P. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022.
- Bottou, L., Curtis, F. E., and Nocedal, J. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Dauphin, Y. N., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., and Bengio, Y. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Advances in neural information processing systems*, 27, 2014.
- Durvasula, S., Zhao, A., Chen, F., Liang, R., Sanjaya, P. K., and Vijaykumar, N. Distwar: Fast differentiable rendering on raster-based rendering pipelines. *arXiv preprint arXiv:2401.05345*, 2023.
- Fan, Z., Wang, K., Wen, K., Zhu, Z., Xu, D., Wang, Z., et al. Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps. *Advances in neural information processing systems*, 37:140138–140158, 2024.
- Fang, G. and Wang, B. Mini-splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision*, pp. 165–181. Springer, 2024.
- Feng, G., Chen, S., Fu, R., Liao, Z., Wang, Y., Liu, T., Hu, B., Xu, L., Pei, Z., Li, H., et al. Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 26652–26662, 2025.
- Girish, S., Gupta, K., and Shrivastava, A. Eagles: Efficient accelerated 3d gaussians with lightweight encodings. In *European Conference on Computer Vision*, pp. 54–71. Springer, 2024.
- Hanson, A., Tu, A., Lin, G., Singla, V., Zwicker, M., and Goldstein, T. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 21537–21546, 2025a.
- Hanson, A., Tu, A., Singla, V., Jayawardhana, M., Zwicker, M., and Goldstein, T. Pup 3d-gs: Principled uncertainty pruning for 3d gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 5949–5958, 2025b.
- Hedman, P., Philip, J., Price, T., Frahm, J.-M., Drettakis, G., and Brostow, G. Deep blending for free-viewpoint image-based rendering. 37(6):257:1–257:15, 2018.
- Höllerin, L., Božič, A., Zollhöfer, M., and Nießner, M. 3dgs-lm: Faster gaussian-splatting optimization with levenberg-marquardt. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 26740–26750, 2025.
- Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- Kerbl, B., Meuleman, A., Kopanas, G., Wimmer, M., Lanvin, A., and Drettakis, G. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Kheradmand, S., Rebain, D., Sharma, G., Sun, W., Tseng, Y.-C., Isack, H., Kar, A., Tagliasacchi, A., and Yi, K. M. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems*, 37:80965–80986, 2024.
- Kingma, D. P. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Knapitsch, A., Park, J., Zhou, Q.-Y., and Koltun, V. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.
- Lan, L., Shao, T., Lu, Z., Zhang, Y., Jiang, C., and Yang, Y. 3dgs2: Near second-order converging 3d gaussian splatting. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pp. 1–10, 2025.

- Lee, J. C., Rho, D., Sun, X., Ko, J. H., and Park, E. Compact 3d gaussian representation for radiance field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21719–21728, 2024.
- Liu, H., Li, Z., Hall, D., Liang, P., and Ma, T. Sophia: A scalable stochastic second-order optimizer for language model pre-training. *arXiv preprint arXiv:2305.14342*, 2023.
- Lu, T., Yu, M., Xu, L., Xiangli, Y., Wang, L., Lin, D., and Dai, B. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20654–20664, 2024.
- Mallick, S. S., Goel, R., Kerbl, B., Steinberger, M., Carrasco, F. V., and De La Torre, F. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, pp. 1–11, 2024.
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.
- Papantonakis, P., Kopanas, G., Kerbl, B., Lanvin, A., and Drettakis, G. Reducing the memory footprint of 3d gaussian splatting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(1):1–17, 2024.
- Park, H., Ryu, G., and Kim, W. Dropgaussian: Structural regularization for sparse-view gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 21600–21609, 2025.
- Pehlivan, H., Camiletto, A. B., Foo, L. G., Habermann, M., and Theobalt, C. Second-order optimization of gaussian splats with importance sampling. *arXiv preprint arXiv:2504.12905*, 2025.
- Rota Bulò, S., Porzi, L., and Kontschieder, P. Revising densification in gaussian splatting. In *European Conference on Computer Vision*, pp. 347–362. Springer, 2024.
- Schaul, T., Zhang, S., and LeCun, Y. No more pesky learning rates. In *Proceedings of the 30th International Conference on Machine Learning*, pp. 343–351, 2013.
- Song, K., Zeng, X., Ren, C., and Zhang, J. City-on-web: Real-time neural rendering of large-scale scenes on the web. In *European Conference on Computer Vision*, pp. 385–402. Springer, 2024.
- Sutton, R. S. Two problems with backpropagation and other steepest-descent learning procedures for networks. In *Proceedings of the annual meeting of the cognitive science society*, volume 8, 1986.
- Wang, X., Yi, R., and Ma, L. Adr-gaussian: Accelerating gaussian splatting with adaptive radius. In *SIGGRAPH Asia 2024 Conference Papers*, pp. 1–10, 2024.
- Yao, Z., Gholami, A., Shen, S., Mustafa, M., Keutzer, K., and Mahoney, M. Adahessian: An adaptive second order optimizer for machine learning. In *proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 10665–10673, 2021.
- Ye, V., Li, R., Kerr, J., Turkulainen, M., Yi, B., Pan, Z., Seiskari, O., Ye, J., Hu, J., Tancik, M., et al. gsplat: An open-source library for gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025.
- Yu, Z., Chen, A., Huang, B., Sattler, T., and Geiger, A. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 19447–19456, 2024.
- Zhang, J., Li, J., Yu, X., Huang, L., Gu, L., Zheng, J., and Bai, X. Cor-gs: sparse-view 3d gaussian splatting via co-regularization. In *European Conference on Computer Vision*, pp. 335–352. Springer, 2024a.
- Zhang, Z., Song, T., Lee, Y., Yang, L., Peng, C., Chellappa, R., and Fan, D. Lp-3dgs: Learning to prune 3d gaussian splatting. *Advances in Neural Information Processing Systems*, 37:122434–122457, 2024b.
- Zhao, H., Weng, H., Lu, D., Li, A., Li, J., Panda, A., and Xie, S. On scaling up 3d gaussian splatting training. In *European Conference on Computer Vision*, pp. 14–36. Springer, 2024.

A. Quantitative Results Per Scene

 Table 2. Results on Deep Blending. Best results are **boldfaced** and second best are underlined.

Method	Scene	Deep Blending									Time (s)	Peak GPU Mem (GB)
		SSIM ↓			PSNR ↑			LPIPS ↓				
		@7k	@15k	@30k	@7k	@15k	@30k	@7k	@15k	@30k		
ADAM	drjohnson	0.841	0.855	0.861	26.60	27.39	27.64	0.368	0.360	0.390	182	5.82
3DGS-LM ¹	drjohnson	0.811	0.837	0.849	24.78	25.87	26.37	0.432	0.393	<u>0.373</u>	3344	47.65
ADAM-TR (Ours)	drjohnson	<u>0.852</u>	0.865	0.869	26.50	<u>27.30</u>	<u>27.56</u>	<u>0.355</u>	<u>0.347</u>	<u>0.378</u>	<u>200</u>	<u>5.90</u>
3DGS ² -TR (Ours) ²	drjohnson	0.855	<u>0.865</u>	<u>0.869</u>	<u>26.54</u>	27.25	27.48	0.353	0.346	0.371	606	6.68
ADAM	playroom	0.849	0.852	0.854	<u>26.70</u>	26.97	27.07	0.376	0.371	0.385	177	5.03
3DGS-LM ¹	playroom	0.809	0.829	0.840	<u>22.66</u>	23.91	24.87	0.443	0.417	0.398	1125	19.01
ADAM-TR (Ours)	playroom	<u>0.859</u>	<u>0.867</u>	<u>0.869</u>	26.63	<u>27.38</u>	<u>27.61</u>	<u>0.359</u>	<u>0.354</u>	<u>0.373</u>	<u>189</u>	<u>5.12</u>
3DGS ² -TR (Ours) ²	playroom	0.863	0.868	0.870	27.28	27.84	27.99	0.354	0.351	0.363	594	5.84

¹ 3DGS-LM results are reported @35, @75, and @150 iterations.

² System implementation is not fully optimized. See Section 6 for details.

 Table 3. Results on Tanks & Temples. Best results are **boldfaced** and second best are underlined.

Method	Scene	Tanks & Temples									Time (s)	Peak GPU Mem (GB)
		SSIM ↓			PSNR ↑			LPIPS ↓				
		@7k	@15k	@30k	@7k	@15k	@30k	@7k	@15k	@30k		
ADAM	train	0.697	0.745	0.758	19.83	<u>20.91</u>	21.21	0.318	0.302	0.362	209	3.45
3DGS-LM ¹	train	0.644	0.688	0.720	19.13	19.92	20.57	0.425	0.377	0.347	2162	28.36
ADAM-TR (Ours)	train	<u>0.740</u>	<u>0.763</u>	<u>0.771</u>	<u>20.21</u>	20.87	<u>21.22</u>	<u>0.297</u>	<u>0.287</u>	<u>0.321</u>	<u>229</u>	<u>3.54</u>
3DGS ² -TR (Ours) ²	train	0.757	0.775	0.781	20.68	21.32	21.62	0.283	0.276	0.302	503	4.43
ADAM	truck	0.750	0.766	0.773	21.56	21.90	22.12	0.301	0.293	0.320	192	2.88
3DGS-LM ¹	truck	0.679	0.747	0.779	20.86	22.25	23.05	0.432	0.355	0.315	2047	22.69
ADAM-TR (Ours)	truck	<u>0.807</u>	<u>0.819</u>	<u>0.824</u>	<u>23.02</u>	<u>23.44</u>	<u>23.62</u>	<u>0.250</u>	<u>0.242</u>	<u>0.268</u>	<u>224</u>	<u>3.00</u>
3DGS ² -TR (Ours) ²	truck	0.816	0.826	0.829	23.55	23.93	24.07	0.240	0.234	0.255	465	3.82

¹ 3DGS-LM results are reported @35, @75, and @150 iterations.

² System implementation is not fully optimized. See Section 6 for details.

Table 4. Results on MipNeRF-360. Best results are **boldfaced** and second best are underlined.

MipNeRF-360												
Method	Scene	SSIM ↓			PSNR ↑			LPIPS ↓			Time (s)	Peak GPU Mem (GB)
		@7k	@15k	@30k	@7k	@15k	@30k	@7k	@15k	@30k		
ADAM	treehill	0.503	0.516	0.520	21.43	21.65	21.69	0.549	0.544	0.558	194	<u>3.47</u>
3DGS-LM ¹	treehill	0.397	0.413	0.430	19.31	19.99	20.52	0.645	0.634	0.617	2648	42.58
ADAM-TR (Ours)	treehill	<u>0.510</u>	<u>0.524</u>	<u>0.529</u>	<u>21.47</u>	<u>21.71</u>	<u>21.75</u>	<u>0.538</u>	<u>0.533</u>	<u>0.552</u>	<u>210</u>	3.46
3DGS ² -TR (Ours) ²	treehill	0.520	0.530	0.533	21.73	21.84	21.84	0.533	0.529	0.543	600	4.26
ADAM	counter	0.852	0.866	0.872	26.49	27.15	27.53	0.279	0.270	0.297	241	7.12
3DGS-LM ¹	counter	0.678	0.709	0.741	20.68	21.95	23.07	0.505	0.480	0.447	3531	61.56
ADAM-TR (Ours)	counter	<u>0.859</u>	<u>0.871</u>	<u>0.876</u>	<u>26.58</u>	<u>27.25</u>	<u>27.59</u>	<u>0.272</u>	<u>0.265</u>	<u>0.290</u>	<u>270</u>	<u>7.24</u>
3DGS ² -TR (Ours) ²	counter	0.865	0.874	0.877	26.91	27.51	27.77	0.268	0.263	0.282	761	8.18
ADAM	stump	0.508	0.517	0.520	22.77	22.83	22.86	0.542	0.538	0.550	182	2.60
3DGS-LM ¹	stump	0.384	0.397	0.416	20.44	20.85	21.30	0.670	0.662	0.643	2121	33.61
ADAM-TR (Ours)	stump	<u>0.512</u>	<u>0.528</u>	<u>0.532</u>	<u>22.78</u>	<u>22.93</u>	<u>22.99</u>	<u>0.529</u>	<u>0.524</u>	<u>0.546</u>	<u>187</u>	<u>2.64</u>
3DGS ² -TR (Ours) ²	stump	0.525	0.534	0.537	22.98	23.08	23.13	0.522	0.518	0.532	589	3.39
ADAM	bonsai	0.893	0.904	0.908	28.37	29.24	29.69	0.285	0.279	0.299	237	8.49
3DGS-LM ¹	bonsai	0.679	0.721	0.760	21.04	22.68	24.06	0.505	0.482	0.450	1921	43.60
ADAM-TR (Ours)	bonsai	<u>0.901</u>	<u>0.911</u>	<u>0.914</u>	<u>28.75</u>	<u>29.67</u>	<u>30.03</u>	<u>0.273</u>	<u>0.268</u>	<u>0.288</u>	<u>278</u>	<u>8.63</u>
3DGS ² -TR (Ours) ²	bonsai	0.905	0.913	0.916	29.11	29.92	30.31	0.270	0.265	0.283	750	9.59
ADAM	bicycle	0.480	0.498	0.504	21.58	21.84	21.95	0.523	0.516	0.539	185	3.70
3DGS-LM ¹	bicycle	0.346	0.360	0.376	18.96	19.47	19.89	0.676	0.666	0.652	2265	42.61
ADAM-TR (Ours)	bicycle	<u>0.494</u>	<u>0.516</u>	<u>0.523</u>	<u>21.70</u>	<u>22.04</u>	<u>22.11</u>	<u>0.505</u>	<u>0.496</u>	<u>0.530</u>	<u>203</u>	<u>3.76</u>
3DGS ² -TR (Ours) ²	bicycle	0.511	0.526	0.532	21.92	22.17	22.24	0.494	0.488	0.510	565	4.53
ADAM	kitchen	0.880	0.891	0.898	27.95	28.64	29.11	0.193	0.185	0.209	271	<u>8.41</u>
3DGS-LM ¹	kitchen	0.592	0.640	0.698	20.94	22.38	23.66	0.490	0.450	0.401	3257	66.58
ADAM-TR (Ours)	kitchen	<u>0.888</u>	<u>0.897</u>	<u>0.902</u>	<u>28.03</u>	<u>28.81</u>	<u>29.20</u>	<u>0.186</u>	<u>0.179</u>	<u>0.200</u>	<u>307</u>	8.35
3DGS ² -TR (Ours) ²	kitchen	0.892	0.899	0.903	28.51	29.13	29.47	0.183	0.177	0.194	761	9.51
ADAM	flowers	0.353	0.362	0.365	18.77	18.88	18.89	0.605	0.602	0.614	187	<u>3.41</u>
3DGS-LM ¹	flowers	0.265	0.277	0.292	17.02	17.51	17.89	0.725	0.711	0.690	2654	48.45
ADAM-TR (Ours)	flowers	<u>0.363</u>	<u>0.374</u>	<u>0.377</u>	<u>18.81</u>	<u>18.94</u>	<u>18.97</u>	<u>0.594</u>	<u>0.590</u>	<u>0.605</u>	<u>202</u>	3.39
3DGS ² -TR (Ours) ²	flowers	0.370	0.377	0.379	18.93	19.02	19.04	0.592	0.589	0.599	555	4.18
ADAM	room	0.870	0.879	0.883	<u>28.80</u>	29.33	29.67	0.309	0.302	0.323	212	8.82
3DGS-LM ¹	room	0.718	0.766	0.797	21.96	23.79	25.32	0.488	0.467	0.438	2696	49.79
ADAM-TR (Ours)	room	<u>0.877</u>	<u>0.886</u>	<u>0.890</u>	<u>28.77</u>	<u>29.49</u>	<u>29.81</u>	<u>0.296</u>	<u>0.290</u>	<u>0.312</u>	<u>238</u>	<u>8.91</u>
3DGS ² -TR (Ours) ²	room	0.881	0.888	0.891	29.00	29.70	30.18	0.292	0.286	0.304	691	9.72
ADAM	garden	0.660	0.678	0.687	23.87	24.11	24.29	0.370	0.363	0.387	202	4.36
3DGS-LM ¹	garden	0.373	0.394	0.428	19.88	20.53	21.23	0.674	0.644	0.595	2327	40.97
ADAM-TR (Ours)	garden	<u>0.664</u>	<u>0.685</u>	<u>0.692</u>	<u>23.98</u>	<u>24.32</u>	<u>24.45</u>	<u>0.356</u>	<u>0.349</u>	<u>0.379</u>	<u>217</u>	<u>4.42</u>
3DGS ² -TR (Ours) ²	garden	0.672	0.688	0.694	24.12	24.37	24.50	0.353	0.346	0.369	591	5.26

¹ 3DGS-LM results are reported @35, @75, and @150 iterations.

² System implementation is not fully optimized. See Section 6 for details.

B. Parameter-wise trust-region bounds based on squared Hellinger distance

We derive parameter-wise trust-region bounds such that when one parameter is updated while all other parameters fixed, the normalized distance between the Gaussian primitive before and after the update guarantees

$$\frac{H^2(G, G')}{\det(S)} < \epsilon.$$

Bound on the mean μ . Let $\mu = (\mu_x, \mu_y, \mu_z)$, $\mu' = (\mu'_x, \mu'_y, \mu'_z)$, $\Delta\mu = \mu' - \mu$. We first let $\Delta\mu_x \neq 0$, $\Delta\mu_y = 0$, $\Delta\mu_z = 0$, then

$$H^2(G, G') = C \left(1 - \exp \left(-\frac{\Delta\mu^T \Sigma^{-1} \Delta\mu}{8} \right) \right) = C \left(1 - \exp \left(-\frac{\Delta\mu_x^2}{8\Sigma_{xx}} \right) \right).$$

Setting $H^2(G, G')/\det(S) < \epsilon$ gives $\alpha \left(1 - \exp \left(-\frac{\Delta\mu_x^2}{8\Sigma_{xx}} \right) \right) < \epsilon$. After rearranging the terms, we have

$$|\Delta\mu_x| < \sqrt{-8\Sigma_{xx} \ln \left(1 - \frac{\epsilon}{\alpha} \right)}.$$

Using similar derivation, we have

$$\begin{aligned} |\Delta\mu_y| &< \sqrt{-8\Sigma_{yy} \ln \left(1 - \frac{\epsilon}{\alpha} \right)}, \\ |\Delta\mu_z| &< \sqrt{-8\Sigma_{zz} \ln \left(1 - \frac{\epsilon}{\alpha} \right)}. \end{aligned}$$

Bound on the scale matrix S . Let

$$S = \begin{bmatrix} S_x & & \\ & S_y & \\ & & S_z \end{bmatrix}, \quad S' = \begin{bmatrix} S'_x & & \\ & S'_y & \\ & & S'_z \end{bmatrix},$$

and define $\Delta S = S' - S$ with differences $\Delta S_x, \Delta S_y, \Delta S_z$.

Note that $C = \alpha \mathcal{C} \det(\Sigma)^{1/2} = \alpha \mathcal{C} \det(S)$, $C' = \alpha \mathcal{C} \det(S')$, we first let $\Delta S_x \neq 0$ while $\Delta S_y = 0$, $\Delta S_z = 0$, and denote $\rho_x = \alpha S_y S_z$. Then

$$H^2(G, G') = \frac{1}{2}(\rho_x S_x + \rho_x S'_x) - \rho_x (S_x S'_x)^{1/2} \frac{(S_x)^{1/2} (S'_x)^{1/2} S_y S_z}{\left(\frac{S_x^2 + S_x'^2}{2} \right)^{1/2} S_y S_z} = \rho_x \frac{S_x + S'_x}{2} - \rho_x \frac{S_x S'_x}{\left(\frac{S_x^2 + S_x'^2}{2} \right)^{1/2}}$$

Since $S'_x = S_x + \Delta S_x$, we have $\frac{d}{d\Delta S_x} H^2 = \rho_x \left(\frac{1}{2} + \frac{-2S_x^2}{(2S_x^2 + \Delta S_x)^2} \right)$, and $\frac{d^2}{d\Delta S_x^2} H^2 = \rho_x (4S_x^2 \cdot (2S_x + \Delta S_x)^{-3})$. We thus can approximate

$$H^2(G, G') \approx \frac{1}{2} \frac{\rho_x}{2S_x} \Delta S_x^2.$$

Setting $H^2(G, G')/\det S < \epsilon$ gives

$$|\Delta S_x| < \sqrt{\frac{4S_x \det(S) \epsilon}{\rho_x}} = \sqrt{\frac{4S_x^2 \epsilon}{\alpha}}.$$

We can similarly obtain

$$|\Delta S_y| < \sqrt{\frac{4S_y^2 \epsilon}{\alpha}}, \text{ and } |\Delta S_z| < \sqrt{\frac{4S_z^2 \epsilon}{\alpha}}.$$

Bound on the opacity α . Let $\rho_\alpha = \det(\Sigma)^{1/2} = \det(S)$, then

$$H^2(G, G') = \rho_\alpha \left(\frac{\alpha + \alpha'}{2} - (\alpha\alpha')^{1/2} \right).$$

Let $\alpha' = \alpha + \Delta\alpha$, then

$$\frac{d}{d\Delta\alpha} H^2 = \rho_\alpha \left(\frac{1}{2} - \frac{1}{2} \alpha (\alpha^2 + \alpha\Delta\alpha)^{-1/2} \right)$$

and

$$\frac{d^2}{d\Delta\alpha^2} H^2 = \rho_\alpha \left(\frac{1}{4} \alpha^2 (\alpha^2 + \alpha\Delta\alpha)^{-3/2} \right).$$

Thus, we can approximate

$$H^2(G, G') \approx \frac{1}{2} \frac{\rho_\alpha}{4\alpha} \Delta\alpha^2.$$

Setting $H^2(G, G')/\det(S) < \epsilon$ gives

$$|\Delta\alpha| < \sqrt{\frac{8\alpha \det(S)\epsilon}{\rho_\alpha}} = \sqrt{8\alpha\epsilon}.$$

Bound on the color $\mathcal{C}$. Let $\mathcal{C} = \mathcal{C}_r + \mathcal{C}_g + \mathcal{C}_b$ and similarly define $\mathcal{C}'$ and $\Delta\mathcal{C}_c = \mathcal{C}'_c - \mathcal{C}_c$ for $c \in \{r, g, b\}$. First, let $\Delta\mathcal{C}_r \neq 0$ and $\Delta\mathcal{C}_g = \Delta\mathcal{C}_b = 0$. Let $\rho_r = \alpha \det(\Sigma)^{1/2} = \alpha \det(S)$, then

$$H^2(G, G') = \rho_r \left(\frac{1}{2} (\mathcal{C}_r + \mathcal{C}'_r) - (\mathcal{C}_r \mathcal{C}'_r)^{1/2} \right).$$

We can approximate

$$H^2(G, G') \approx \frac{1}{2} \frac{\rho_r}{4\mathcal{C}_r} \Delta\mathcal{C}_r^2.$$

Setting $H^2(G, G')/\det(S) < \epsilon$ gives

$$|\Delta\mathcal{C}_r| < \sqrt{\frac{8\mathcal{C}_r \det(S)\epsilon}{\rho_r}} = \sqrt{\frac{8\mathcal{C}_r\epsilon}{\alpha}}.$$

Using similar derivation, we also have

$$|\Delta\mathcal{C}_g| < \sqrt{\frac{8\mathcal{C}_g\epsilon}{\alpha}}, \quad |\Delta\mathcal{C}_b| < \sqrt{\frac{8\mathcal{C}_b\epsilon}{\alpha}}.$$

Bound on the rotation R . The rotation matrix R is parameterized by a unnormalized quaternion $\tilde{q} = (q_x, q_y, q_z, q_w)$ with squared norm $r^2 = \|\tilde{q}\|^2 = q_x^2 + q_y^2 + q_z^2 + q_w^2$, yielding

$$R = \begin{bmatrix} 1 - \frac{2(q_y^2 + q_z^2)}{r^2} & \frac{2(q_x q_y - q_w q_z)}{r^2} & \frac{2(q_x q_z + q_w q_y)}{r^2} \\ \frac{2(q_x q_y + q_w q_z)}{r^2} & 1 - \frac{2(q_x^2 + q_z^2)}{r^2} & \frac{2(q_y q_z - q_w q_x)}{r^2} \\ \frac{2(q_x q_z - q_w q_y)}{r^2} & \frac{2(q_y q_z + q_w q_x)}{r^2} & 1 - \frac{2(q_x^2 + q_y^2)}{r^2} \end{bmatrix}.$$

We can similarly construct R' , which is parameterized by another unnormalized quaternion $q' = (q'_x, q'_y, q'_z, q'_w)$ with squared norm $r'^2 = \|q'\|^2 = q'^2_x + q'^2_y + q'^2_z + q'^2_w$.

Since $\Sigma = R^T S^T S R$, $\Sigma' = R'^T S^T S R'$ and R, R' are orthogonal, we have $\det(\Sigma) = \det(\Sigma') = \det(S)^2$. Let $\Delta R = R^{-1} R'$ also be an orthogonal rotation matrix. Then

$$H^2(G, G') = C \left(1 - \frac{\det(S)}{\det(\frac{\Sigma + \Sigma'}{2})^{\frac{1}{2}}} \right) = C \left(1 - \frac{\det(S)}{\det(\frac{S^2 + \Delta R^T S^2 \Delta R}{2})^{\frac{1}{2}}} \right).$$

Setting $H^2(G, G')/\det(S) < \epsilon$ and rearranging the terms gives

$$1 - \frac{\det(S)}{\det\left(\frac{S^2 + \Delta R^T S^2 \Delta R}{2}\right)^{\frac{1}{2}}} < \frac{\epsilon}{\alpha}.$$

To satisfy the above inequality, it is sufficient that $\det\left(\frac{S^2 + \Delta R^T S^2 \Delta R}{2}\right) < \left(\frac{\det(S)}{1 - \epsilon/\alpha}\right)^2$, which is sufficient if $\det\left(S^2\left(\frac{I + S^{-2}\Delta R^T S^2 \Delta R}{2}\right)\right) < \left(\frac{\det(S)}{1 - \epsilon/\alpha}\right)^2$. Since for two matrices A and B , $\det(AB) = \det(A)\det(B)$, we only need $\det\left(\frac{I + S^{-2}\Delta R^T S^2 \Delta R}{2}\right) < \left(\frac{1}{1 - \epsilon/\alpha}\right)^2$, equivalently,

$$\det\left(I + \left(-I + \frac{I + S^{-2}\Delta R^T S^2 \Delta R}{2}\right)\right) < \left(\frac{1}{1 - \epsilon/\alpha}\right)^2.$$

By the relation $\det(I + X) \leq \exp(\text{tr}(X))$, it is sufficient to have

$$\text{tr}\left(\frac{I + S^{-2}\Delta R^T S^2 \Delta R}{2} - I\right) < -2 \ln\left(1 - \frac{\epsilon}{\alpha}\right).$$

By the linearity of trace, we only need to show

$$\text{tr}(S^{-2}\Delta R^T S^2 \Delta R) - 3 < -4 \ln\left(1 - \frac{\epsilon}{\alpha}\right). \quad (7)$$

Next, we investigate the approximation of $\text{tr}(S^{-2}\Delta R^T S^2 \Delta R)$. For an unnormalized quaternion $\tilde{q} = (q_x, q_y, q_z, q_w)$ with $r^2 = \|\tilde{q}\|^2$, the unnormalized rotation matrix is

$$\tilde{R}(\tilde{q}) = \begin{bmatrix} r^2 - 2(q_y^2 + q_z^2) & 2q_x q_y - 2q_w q_z & 2q_x q_z + 2q_w q_y \\ 2q_x q_y + 2q_w q_z & r^2 - 2(q_z^2 + q_x^2) & 2q_y q_z - 2q_w q_x \\ 2q_x q_z - 2q_w q_y & 2q_y q_z + 2q_w q_x & r^2 - 2(q_x^2 + q_y^2) \end{bmatrix}$$

which can be normalized as $R = \tilde{R}/r^2$.

Let the update to the unnormalized quaternion be $\Delta q = (\Delta q_x, \Delta q_y, \Delta q_z, \Delta q_w)$. We consider an update in the direction of Δq with a step size a , denoted as $q' = \tilde{q} + a\Delta q$ with $r'^2 = (q_x + a\Delta q_x)^2 + (q_y + a\Delta q_y)^2 + (q_z + a\Delta q_z)^2 + (q_w + a\Delta q_w)^2$

Let $R' = \tilde{R}(q')/r'^2 = R\Delta R = R(I + E)$, where $R, \Delta R, R'$ are orthogonal, and $E = R^T R' - I = R^T \tilde{R}'/r'^2 - I$. Then with the relation $\text{tr}(S^{-2}\Delta R^T S^2 \Delta R) = \text{tr}(S^{-1}\Delta R^T S S \Delta R S^{-1}) = \|S\Delta R S^{-1}\|_F^2 = \|S(I + E)S^{-1}\|_F^2$, we have

$$\begin{aligned} \frac{\partial}{\partial x} \|S(I + E)S^{-1}\|_F^2 &= 2\text{tr}((S(I + E)S^{-1})^T (S\partial_x E S^{-1})), \\ \frac{\partial^2}{\partial x^2} \|S(I + E)S^{-1}\|_F^2 &= 2\|S\partial_x E S^{-1}\|_F^2 + 2\text{tr}((S(I + E)S^{-1})^T (S\partial_x^2 E S^{-1})), \end{aligned}$$

where the partial derivative to x is short for the partial derivative to q_x . We have similar formulas for the partial derivative to q_y, q_z, q_w . For the variable $q_c \in \{q_x, q_y, q_z, q_w\}$, it can be computed that $\partial_c E = R^T(r^{-2}\partial_c \tilde{R} - 2q_c r^{-4}\tilde{R})$, and

$$\begin{aligned} \partial_c^2 E &= R^T(-2q_c r^{-4}\partial_c \tilde{R} + r^{-2}\partial_c^2 \tilde{R} + 6q_c^2 r^{-6}\tilde{R}_c - 2q_c q^{-4}\partial_c \tilde{R} - 2q^{-4}\tilde{R} + 2q_c^2 r^{-6}\tilde{R}) \\ &= R^T(-2q_c r^{-4}\partial_c \tilde{R} + r^{-2}\partial_c^2 \tilde{R} + 8q_c^2 r^{-6}\tilde{R} - 2q_c r^{-4}\partial_c \tilde{R} - 2r^{-4}\tilde{R}). \end{aligned}$$

Denote $T(\Delta q) = \|S(I + E)S^{-1}\|_F^2$, then when $\Delta q_x \neq 0, \Delta q_y = 0, \Delta q_z = 0, \Delta q_w = 0$, we have $T(\Delta q_x) = 3, T'(\Delta q_x) = 0, \beta_x = T''(\Delta q_x) = 2\|S\partial_x E S^{-1}\|_F^2 + 2\text{tr}(\partial_x^2 E)$. We thus can approximate $\text{tr}(S^{-2}\Delta R^T S^2 \Delta R)$ by its second-order Taylor expansion and obtain the approximation of (7) as $3 + \frac{1}{2}\beta_x(\Delta q_x)^2 - 3 < -4 \ln(1 - \epsilon/\alpha)$. Rearranging the terms, we obtain

$$|\Delta q_x| < \sqrt{-8/\beta_x \ln(1 - \epsilon/\alpha)}.$$

Using similar derivation, we also obtain the bounds

$$|\Delta q_y| < \sqrt{-8/\beta_y \ln(1 - \epsilon/\alpha)}, |\Delta q_z| < \sqrt{-8/\beta_z \ln(1 - \epsilon/\alpha)}, \text{ and } |\Delta q_w| < \sqrt{-8/\beta_w \ln(1 - \epsilon/\alpha)}.$$